

PX4 version: 1.16

飞控: TS IFC1

任务:

- 1.针对PX4, 1.16版本, QGC5.0.8, 进行固件剪裁和bug修复工作, 主要理解软件整体架构(操作系统、软件整体逻辑、MAVlink通信交互), 飞控算法部分内容(包括外环、内环、ESC控制)、感知导航部分(EKF), 整理出一个整体的代码解析readme, 对固件对无关部分可以进行剪裁(其他模式的代码, 不用的传感器的代码等, 后续整理个详细需求);
- 2.搭建一个算法的HIL仿真系统, 可实现算法的在线仿真, 对剪裁的固件进行测试仿真(开学前有一版本);
- 3.搭建一个内环飞控测试台, 可实现sim-to-real的算法验证;

一.固件裁剪

固件裁剪主要在/PX4-Autopilot/boards/px4/fmu-v6x/default.px4board文件中进行。通过配置模块的启动与否来控制其代码是否编译, 以达到固件裁剪的目的。

/PX4-Autopilot/boards/px4/fmu-v6x/default.px4board

展示修改的部分:

```
CONFIG_COMMON_DIFFERENTIAL_PRESSURE=y    #压差传感器开启
CONFIG_DRIVERS_MS4525=y                    #空速计开启
CONFIG_DRIVERS_GNSS_SEPTENTRIO=n          #关闭这个GNSS
CONFIG_DRIVERS_GPS=y                       #适配NEO-M9N-00B GNSS
CONFIG_DRIVERS_IMU_BOSCH_BMI088=y         #适配BMI088
CONFIG_DRIVERS_IMU_INVENSENSE_ICM20602=n  #关闭
CONFIG_DRIVERS_IMU_INVENSENSE_ICM20649=n  #关闭
CONFIG_DRIVERS_IMU_INVENSENSE_ICM20948=n  #关闭
CONFIG_DRIVERS_IMU_INVENSENSE_ICM42670P=n #关闭
CONFIG_DRIVERS_IMU_INVENSENSE_ICM42688P=y #适配ICM42688-P
CONFIG_DRIVERS_IMU_INVENSENSE_ICM45686=n  #关闭
CONFIG_DRIVERS_IMU_INVENSENSE_IIM42652=n  #关闭
CONFIG_MODULES_FW_ATT_CONTROL=y            #开启固定翼模块
CONFIG_MODULES_FW_AUTOTUNE_ATTITUDE_CONTROL=y #开启固定翼模块
CONFIG_MODULES_FW_POS_CONTROL=y           #开启固定翼模块
CONFIG_MODULES_FW_RATE_CONTROL=y          #开启固定翼模块
CONFIG_MODULES_MC_ATT_CONTROL=y           #开启多旋翼模块
CONFIG_MODULES_MC_AUTOTUNE_ATTITUDE_CONTROL=y #开启多旋翼模块
CONFIG_MODULES_MC_HOVER_THRUST_ESTIMATOR=y #开启多旋翼模块
CONFIG_MODULES_MC_POS_CONTROL=y           #开启多旋翼模块
CONFIG_MODULES_MC_RATE_CONTROL=y          #开启多旋翼模块
CONFIG_MODE_NAVIGATOR_VTOL_TAKEOFF=n      #关闭VTOL的相关模块
```

```
CONFIG_MODULES_VTOL_ATT_CONTROL=n
```

```
#关闭VTOL的相关模块
```

无人车、无人潜艇等模块是默认关闭的，可以不用设置。如果要设置，可以在/PX4-Autopilot/boards/px4/fmu-v6x/default.px4board文件中显式开启，也可以在PX4/PX4-Autopilot/src/modules中对应模块的Kconfig文件中开启或者关闭。

注意，在PX4代码中不要使用中文注释，会报错！

二.rcS脚本详细解析(对照rcS文件阅读此章节)

- 飞控硬件上电并且完成Bootloader引导后，由实时操作系统NuttX调用第一个启动脚本：PX4-Autopilot/ROMFS/px4fmu_common/init.d/rcS，该脚本主要完成如下任务：

- 脚本环境与调试控制

```
#!/bin/sh #指定脚本解释器为系统的shell程序
set +e #设置“忽略错误继续执行”模式。这意味着即使某行命令运行失败，脚本也不会立即退出，确保系统能尽可能多的初始化硬件
#set -x #此行被注释掉，若启用，系统会打印出脚本执行的每一条指令，通常用于底层调试
```

- 基础路径变量设置

```
set R / #定义根目录变量${R}为 /
set FCONFIG /fs/microsd/etc/config.txt #定义用户自定义配置文件的路径，位于 SD 卡中
set FEXTRAS /fs/microsd/etc/extras.txt #定义额外启动脚本的路径，用于加载用户自定义模块
set FRC /fs/microsd/etc/rc.txt #定义备用启动脚本路径，若此文件存在，通常会改变默认启动流程
set IOFW "/etc/extras/px4_io-v2_default.bin" #指定PX4IO协同处理器的固件镜像路径，用于后续的固件版本校验或更新
```

- 系统状态与日志参数初始化

```
set LOGGER_ARGS "" #初始化日志记录器（Logger）参数为空字符串，在脚本后期启动 logger 进程时，系统会将用户在 config.txt 或其他地方定义的额外参数累加到这个变量中。如果保持为空，则使用系统默认的日志配置
set LOGGER_BUF 8 #设置日志缓存大小，默认为8KB
set PARAM_FILE "" #初始化主参数文件路径变量，这是一个占位符。在脚本后续检测到存储设备可用后，会将实际的参数文件路径赋值给它，随后通过param select命令进行加载。
set PARAM_BACKUP_FILE "" #初始化参数备份文件路径变量，同样是占位符。当 SD 卡挂载成功后，该变量通常会被设置为 "/fs/microsd/parameters_backup.bson"。这样当主参数文件损坏时，系统知道去哪里找备份进行恢复
set RC_INPUT_ARGS "" #初始化遥控器输入（RC Input）启动参数，用于存储传递给 rc_input 驱动的特定指令。例如，某些特殊的接收机协议需要额外的配置参数，这些参数会暂存在这里，直到
```

```
执行 rc_input start $RC_INPUT_ARGS 时被调用
set STORAGE_AVAILABLE no #初始化存储可用状态为no，后续挂载 SD 卡成功后会改为yes
set SDCARD_EXT_PATH /fs/microsd/ext_autostart #指定SD卡上存放外部机架配置文件的文件夹路径
set SDCARD_FORMAT no #初始化“是否格式化SD卡”标志为no
set STARTUP_TUNE 1 #置启动提示音类型，1 通常对应标准的系统启动音
set VEHICLE_TYPE none #初始化机型类型为none，直到后续加载具体机架脚本（如多旋翼或固定翼）后才会改变
#-----
set PARAM_DEFAULTS_VER 1 #设置机架参数默认版本号为1。如果开发者修改了机架默认参数并希望强制用户更新，会通过提高此版本号来触发参数重置

ver all #这是一个可执行命令，调用后会在控制台打印当前系统的详细版本信息
```

- 存储卡挂载:脚本首先通过检测块设备 /dev/mmcsd0 来尝试挂载 SD 卡到 /fs/microsd 路径。如果挂载成功且没有发现 .format 文件，则将 STORAGE_AVAILABLE 变量设置为 yes。若硬件设备不存在，则会尝试查询 MTD 参数分区作为替代方案
- 挂载SD卡部分技术栈为FAT32

- 尝试挂载SD卡

```
if [ -b "/dev/mmcsd0" ]
then
    if mount -t vfat /dev/mmcsd0 /fs/microsd
    then
        if [ -f "/fs/microsd/.format" ]
        then
            echo "INFO [init] format /dev/mmcsd0 requested (/fs/microsd/.format)"
            set SDCARD_FORMAT yes
            rm /fs/microsd/.format
            umount /fs/microsd

        else
            set STORAGE_AVAILABLE yes
        fi
    fi
fi
```

- 执行格式化逻辑：如果初次挂载失败或检测到格式化标记，脚本会执行以下格式化并重新挂载的操作

```
if [ $STORAGE_AVAILABLE = no -o $SDCARD_FORMAT = yes ]
then
    echo "INFO [init] formatting /dev/mmcsd0"
    set STARTUP_TUNE 15 # tune 15 = SD_ERROR (overridden to SD_INIT if format
+ mount succeeds)

    if mkfatfs -F 32 /dev/mmcsd0
    then
        echo "INFO [init] card formatted"
```

```

        if mount -t vfat /dev/mmc0 /fs/microsd
        then
            set STORAGE_AVAILABLE yes
            set STARTUP_TUNE 14 # tune 14 = SD_INIT
        else
            echo "ERROR [init] card mount failed"
        fi
    else
        echo "ERROR [init] format failed"
    fi
fi

```

- MTD备选方案与挂载逻辑结束：如果 /dev/mmc0 设备不存在，脚本会尝试通过 mft 查询其他可用的存储分区。

```

else
    # Is there a device mounted for storage
    if mft query -q -k MTD -s MTD_PARAMETERS -v /mnt/microsd
    then
        set STORAGE_AVAILABLE yes
    fi
fi

```

- 异常处理：检查是否存在硬件故障(hardfault)产生的残留日志，如果存在崩溃日志，将启动提示音设为“错误音”(ERROR_TURN)来提醒用户，并且尝试将内存中的崩溃日志写入到SD卡中，方便后续导出分析。

```

if [ $STORAGE_AVAILABLE = yes ]
then
    if hardfault_log check
    then
        set STARTUP_TUNE 2 # tune 2 = ERROR_TUNE
        if hardfault_log commit
        then
            hardfault_log reset
        fi
    fi

    # Check for an update of the ext_autostart folder, and replace the old one
    with it
    if [ -e /fs/microsd/ext_autostart_new ]
    then
        echo "Updating external autostart files"
        rm -r $SDCARD_EXT_PATH
        mv /fs/microsd/ext_autostart_new $SDCARD_EXT_PATH
    fi

    set PARAM_FILE /fs/microsd/params
    set PARAM_BACKUP_FILE "/fs/microsd/parameters_backup.bson"

```

```
fi
```

- 参数加载与机架匹配
- 代码首先检查 SD 卡上是否存在一个名为`rc.txt`的完全替代脚本,如果有,脚本会直接执行它,并且跳过后面所有的默认启动逻辑,这通常用于底层调试,或者在文件系统损坏无法启动标准流程时进行救急

```
if [ -f $FRC ]
then
    .$FRC
```

- 如果没有替代脚本,则加载`rc.filepaths`,获取系统中各种配置文件的标准路径。

```
else
```

Load param file location from kconfig #这里说从kconfig读取参数,其实就是我们裁剪固件的文件以及drivers和modules文件中的kconfig文件决定。这里解释了我们裁剪配置外设是怎么实现的。

```
. ${R}etc/init.d/rc.filepaths #这个文件就是编译固件时由构建系统生成的
```

- 确存储储在芯片内部 MTD 分区中的工厂校准数据(陀螺仪、加速度计的出厂校准值)是完好无损的,防止飞控加载错误的校准数据导致飞行事故

```
# Check if /fs/mtd_params is a valid BSON file
if ! bsondump docsize /fs/mtd_caldata
then
    echo "New /fs/mtd_caldata size is:"
    bsondump docsize /fs/mtd_caldata
fi
```

- 加载工厂校准数据

```
#
# Load parameters.
#
# if the board has a storage for (factory) calibration data
if mft query -q -k MTD -s MTD_CALDATA -v /fs/mtd_caldata
then
    param load /fs/mtd_caldata
fi
```

- 加载用户主参数

```
param select $PARAM_FILE
```

- 如果主参数文件损坏（导入失败），脚本会执行以下一系列急救措施：
- 在控制台打印错误信息
- 将开机提示音设置为错误音，用声音警告用户

```
if ! param import
then
    echo "ERROR [init] param import failed"
    set STARTUP_TUNE 2 # tune 2 = ERROR_TUNE
```

- 尝试打印损坏文件的结构以便调试

```
bsondump $PARAM_FILE
```

- 如果 SD 卡可用，将损坏的参数文件复制一份保存，命名为 `param_import_fail.bson`，供开发者后续分析原因

```
if [ -d "/fs/microsd" ]
then
    # try to make a backup copy
    cp $PARAM_FILE /fs/microsd/param_import_fail.bson
```

- 尝试备份恢复,并且将内核启动日志输出到SD卡上的 `param_import_fail.txt` 文件中
- 这里是备份恢复时，是脚本在控制，脚本知道路径在哪，直接指给飞控看。

```
# try importing from backup file
if [ -f $PARAM_BACKUP_FILE ]
then
    echo "[init] importing from parameter backup"

    # dump current backup file contents for comparison
    bsondump $PARAM_BACKUP_FILE

    param import $PARAM_BACKUP_FILE

    # overwrite invalid $PARAM_FILE with backup
    cp $PARAM_BACKUP_FILE $PARAM_FILE
fi

param status

dmesg >> /fs/microsd/param_import_fail.txt &
```

```
fi
fi
```

- 如果SD卡可用，就告诉系统将参数的备份文件路径设置为 `$PARAM_BACKUP_FILE`，之后保存参数的时候，也会在这个地址下保存一个备份。
- 这里的意思是说，在保存备份的时候是飞控系统在后台自动控制，所以需要提前注册好路径，飞控才知道往哪写。

```
if [ $STORAGE_AVAILABLE = yes ]
then
    param select-backup $PARAM_BACKUP_FILE
fi
```

- 以太网硬件检测与初始化（如果支持以太网的话）

```
if mft query -q -k MFT -s MFT_ETHERNET -v 1
then
    netman update -i eth0
fi
```

- 在重置飞控大部分参数（如 PID、安全设置等）的同时，保留最关键的校准数据和机架配置，从而避免重置后必须重新进行繁琐的传感器和遥控器校准

```
# To trigger a parameter reset during boot SYS_AUTOCONFIG was set to 1 before
if param greater SYS_AUTOCONFIG 0
then
    # Reset parameters except airframe, parameter version, RC calibration,
    sensor calibration, flight modes, total flight time, flight UUID
```

- 这些参数在重置飞控时被保留
- `SYS_AUTOSTART`机架类型
- `SYS_PARAM_VER`参数版本号，用于防止重复触发重置
- `RC*`遥控器校准数据。保留了你的摇杆最大/最小值和通道映射，不需要重做 RC 校准
- `CAL_*`传感器校准数据。最关键的部分！保留了加速度计、陀螺仪、磁力计和水平校准数据
- `COM_FLTMODE*`飞行模式开关设置。保留了你习惯的“定点”、“自稳”等开关映射
- `LND_FLIGHT* TC_* COM_FLIGHT*`飞行统计数据。保留了总飞行时间、飞行次数和唯一的飞行 UUID，用于寿命记录

```
param reset_all SYS_AUTOSTART SYS_PARAM_VER RC* CAL_* COM_FLTMODE*
LND_FLIGHT* TC_* COM_FLIGHT*
fi
```

- PX4 启动配置的分层加载机制中的第一层——架构级默认配置

- [PX4-Autopilot/platforms/nutttx/init/stm32h7/rc.board_arch_defaults](#)
- 在加载具体机型参数之前，先加载STM32H7架构通用的默认配置
- 将 EKF2（估计器）的多 IMU 融合模式设为 3（通常指多路并发融合或更激进的冗余策略）。
- 开启实时频谱分析 (FFT)
- 开启 PID 自动调参，开启此功能后，可以在QGC中绑定一个RC的按键，按下这个按键，STM32H7会故意向电机发送瞬间的扰动指令，让PX4内部的PID自动调参，获得更优化的参数。——系统辨识（System Identification）
- 开启高级 CAN 总线支持
- 增大日志缓存

```
#
# Optional board architecture defaults: rc.board_arch_defaults
#
set BOARD_ARCH_RC_DEFAULTS ${R}etc/init.d/rc.board_arch_defaults
if [ -f $BOARD_ARCH_RC_DEFAULTS ]
then
    echo "Board architecture defaults: ${BOARD_ARCH_RC_DEFAULTS}"
    . $BOARD_ARCH_RC_DEFAULTS
fi
unset BOARD_ARCH_RC_DEFAULTS
```

-板级硬件默认配置 -开网口: 默认配置好了以太网连接 QGC -选芯片: 告诉系统电源模块是 INA226。 -管温度: 开启 IMU 加热 -定版本: 解决不同批次硬件的传感器 ID 冲突 -具体看下面这个文件 -文件位置[PX4-Autopilot/boards/ark/fmu-v6x/init/rc.board_defaults](#)

```
#
# Optional board defaults: rc.board_defaults
#
set BOARD_RC_DEFAULTS ${R}etc/init.d/rc.board_defaults
if [ -f $BOARD_RC_DEFAULTS ]
then
    echo "Board defaults: ${BOARD_RC_DEFAULTS}"
    . $BOARD_RC_DEFAULTS
fi
unset BOARD_RC_DEFAULTS
```

- 机架配置加载
- 根据设定的机架ID，从FLASH中找到对应的配置文件并运行
- 如果flash中没有会去sd卡中找
- 如果都没有则报错，会有蜂鸣器提示音
- `VEHICLE_TYPE`来自对应的机架文件

```
# Load airframe configuration based on SYS_AUTOSTART parameter
if ! param compare SYS_AUTOSTART 0
then
    # rc.autostart directly run the right airframe script which sets the
```

```

VEHICLE_TYPE
# Look for airframe in ROMFS
. ${R}etc/init.d/rc.autostart

if [ ${VEHICLE_TYPE} == none ]
then
    # Use external startup file
    if [ $STORAGE_AVAILABLE = yes ]
    then
        . ${R}etc/init.d/rc.autostart_ext
    else
        echo "ERROR [init] SD card not mounted - can't load external
airframe"
    fi
fi

if [ ${VEHICLE_TYPE} == none ]
then
    echo "ERROR [init] No airframe file found for SYS_AUTOSTART value"
    param set SYS_AUTOSTART 0
    tune_control play error
fi
fi

```

- 当刷写了新版本的固件时，强制系统进行一次重启和参数清理，以防止旧版本的参数在新固件上导致错误或炸机
- **SYS_AUTOCONFIG 1**，重新上电后会执行数据清除

```

# Check parameter version and reset upon airframe configuration version
mismatch.
# Reboot required because "param reset_all" would reset all "param set" lines
from airframe.
if ! param compare SYS_PARAM_VER ${PARAM_DEFAULTS_VER}
then
    echo "Switched to different parameter version. Resetting parameters."
    param set SYS_PARAM_VER ${PARAM_DEFAULTS_VER}
    param set SYS_AUTOCONFIG 1
    param save
    reboot
fi

```

- 启动蜂鸣器驱动**tone_alarm**

```

#
# Start the tone_alarm driver.
# Needs to be started after the parameters are loaded (for CBRK_BUZZER).
#
tone_alarm start

```

- 加载航点数据'
- 如果SYS_DM_BACKEND 1, 则从RAM中读取航点, 快
- 如果SYS_DM_BACKEND 0, 则从SD卡中读取航点, 没那么快
- dataman即为datamanager

```
#
# Waypoint storage.
# REBOOTWORK this needs to start in parallel.
#
if param compare -s SYS_DM_BACKEND 1
then
    dataman start -r
else
    if param compare SYS_DM_BACKEND 0
    then
        # dataman start default
        dataman start
    fi
fi
```

- 启动事件发送器
- 外设 (Hardware) -> 驱动程序 (Driver) -> uORB (内部消息) -> send_event (广播员) -> MAVLink (QGC 弹窗) / 蜂鸣器 (滴滴响)

```
#
# Start the socket communication send_event handler.
#
send_event start
```

- 启动负载监控器
- cpuload
- 每隔一段时间检查一下CPU、RAM的占用率

```
#
# Start the resource load monitor.
#
load_mon start
```

- 启动状态指示灯
- rgbled通常指板载的PWM控制的LED
- 剩下的为特定I2C LED驱动芯片型号

```
#
# Start system state indicator.
#
rgbled start -X -q
```

```
rgbled_ncp5623c start -X -q
rgbled_lp5562 start -X -q
rgbled_is31fl3195 start -X -q
```

- 加载用户自定义配置
- 允许在不重新编译固件的情况下，通过 SD 卡里的文件来修改飞控的启动行为
- 由这句`set FCONFIG /fs/microsd/etc/config.txt`设置了该文件

```
#
# Override parameters from user configuration file.
#
if [ -f $FCONFIG ]
then
    echo "Custom: ${FCONFIG}"
    . $FCONFIG
fi
```

- 启动传感器系统
- 通过判断`SYS_HITL`来区分是在进行HITL还是真实飞行
- 如果`SYS_HITL 0`，则为HITL仿真环境，意味着飞控不再读取真实的陀螺仪/加速度计、GPS数据，而是准备接收来自电脑（仿真器）发过来的“假数据”，这里启动的传感器都是模拟的传感器

```
#
# Sensors System (start before Commander so Preflight checks are properly
run).
#
if param greater SYS_HITL 0
then
    sensors start -h

    # disable GPS
    param set GPS_1_CONFIG 0

    # start the simulator in hardware if needed
    if param compare SYS_HITL 2
    then
        simulator_sih start
        sensor_baro_sim start
        sensor_mag_sim start
        sensor_gps_sim start
        sensor_agp_sim start
    fi
```

- 如果`SYS_HITL 1`，则为真实环境
- 先加载板载传感器配置，告诉系统传感器都在什么总线上
- 第二步加载通用传感器脚本，初始化一些标准的驱动逻辑

- 第三步启动电池监控，如果BAT1_SOURCE 2，则启动 esc_battery 驱动，从数字电调读取电压电流；如果BAT1_SOURCE 1，那么就启动battery_status。这是最常见的模拟电压电流计（ADC）驱动，也就是我们平时用的那种电源模块
- 第四步正式启动传感器，前面只是加载驱动，现在开始启动他们。
- 启动后台进程，开始读取所有传感器的数据，进行滤波、校准，然后发布到 uORB 总线上给姿态解算模块使用

```

else
    #
    # board sensors: rc.sensors
    #
    set BOARD_RC_SENSORS ${R}etc/init.d/rc.board_sensors
    if [ -f $BOARD_RC_SENSORS ]
    then
        echo "Board sensors: ${BOARD_RC_SENSORS}"
        . $BOARD_RC_SENSORS
    fi
    unset BOARD_RC_SENSORS

    . ${R}etc/init.d/rc.sensors

    if param compare -s BAT1_SOURCE 2
    then
        esc_battery start
    fi

    if ! param compare BAT1_SOURCE 1
    then
        battery_status start
    fi

    sensors start
fi

```

- 根据参数选择并启动状态估计器
- 飞控需要知道自己“在哪里”和“姿态如何”，这依靠估计器算法
- PX4 提供了三种算法
- EKF2（扩展卡尔曼滤波）--主流
- LPE（局部位置估计器）
- Attitude Q（Q 姿态估计器）

```

#
# state estimator selection
#
if param compare -s EKF2_EN 1
then
    ekf2 start &
fi

if param compare -s LPE_EN 1

```

```

then
    local_position_estimator start
fi

if param compare -s ATT_EN 1
then
    attitude_estimator_q start
fi

```

- PX4IO 协处理器 (IO Co-processor) 的固件检查、自动更新与启动
- 一些高级飞控采用双芯片架构，分别称为FMU(Flight Management Unit)和IO(Input/Output)
- FMU是主芯片，负责复杂的姿态解算、导航和通讯
- IO是协处理器专门负责输出 PWM 信号给电机、读取遥控器信号 (RC Input) 以及处理硬件安全开关 (Safety Switch)
- 小飞控可能是单芯片，主控芯片负责所有功能
- 这段代码先判断是否由IO芯片，如果有，则检查IO芯片的固件和FMU的固件版本是否匹配，如果不匹配，FMU会刷新IO的固件，使版本匹配，最后启动IO，让其控制电机。

```

#
# px4io
#
if px4io supported
then
# Check if PX4IO present and update firmware if needed.
    if [ -f $IOFW ]
    then
        if ! px4io checkcrc ${IOFW}
        then
            # tune Program PX4IO
            tune_control play -t 16 # tune 16 = PROG_PX4IO

            if px4io update ${IOFW}
            then
                usleep 10000
                tune_control stop
                if px4io checkcrc ${IOFW}
                then
                    tune_control play -t 17 # tune 17 = PROG_PX4IO_OK
                else
                    tune_control play -t 18 # tune 18 = PROG_PX4IO_ERR
                fi
            else
                tune_control stop
            fi
        fi
    fi

    if ! px4io start
    then
        echo "PX4IO start failed"
        set STARTUP_TUNE 2 # tune 2 = ERROR_TUNE
    fi

```

```
fi
fi
```

- 启动 IMU（惯性测量单元）的恒温加热驱动
- 陀螺仪和加速度计对温度非常敏感
-

```
# Heater driver for temperature regulated IMUs.
# The heater needs to start after px4io.
if param compare -s SENS_EN_THERMAL 1
then
    heater start
fi
```

- 处理遥控器（Remote Controller）输入的信号
- 发布manual_control_setpoint话题

```
#
# RC update (map raw RC input to calibrate manual control)
# start before commander
#
rc_update start
manual_control start
```

- 启动相机控制、精准时间同步和转速测量
- 在飞控分配引脚给电机之前，先检查用户是否开启了航测拍照、时间同步或转速测量功能。如果有，就先把对应的引脚锁定给这些驱动使用，避免冲突。

```
# Start camera trigger, capture and PPS before pwm_out as they might access
# pwm pins
if param greater -s TRIG_MODE 0
then
    camera_trigger start
    camera_feedback start
fi
# PPS capture driver
if param greater -s PPS_CAP_ENABLE 0
then
    pps_capture start
fi
# RPM capture driver
if param greater -s RPM_CAP_ENABLE 0
then
    rpm_capture start
fi
# Camera capture driver
```

```

if param greater -s CAM_CAP_FBACK 0
then
    if camera_capture start
    then
        camera_capture on
    fi
fi

```

- 启动指挥官和动力输出
- 让飞控开始处理飞行逻辑，并准备好控制电机
- `SYS_HITL 0`，即为HITL仿真模式
- `SYS_HITL 1`，即为真实飞行模式

```

#
# Commander
#
if param greater SYS_HITL 0
then
    commander start -h

    if ! pwm_out_sim start -m hil
    then
        tune_control play error
    fi

else
    commander start

    dshot start
    pwm_out start
fi

```

- 启动飞行控制核心算法
- 根据`VEHICLE_TYPE`启动对应的飞行控制软件模块
- 如果是mc，它会启动`mc_att_control`（姿态控制器）、`mc_pos_control`（位置控制器）和`mc_hover_thrust_estimator`（悬停油门估计器）
- 如果是fw，它会启动`fw_att_control`（固定翼姿态控制）和`fw_pos_control_l1`（L1 导航算法）

```

#
# Configure vehicle type specific parameters.
# Note: rc.vehicle_setup is the entry point for all vehicle type specific
setup.
. ${R}etc/init.d/rc.vehicle_setup

```

- 罗盘偏差估计器
- 启动一个不需要转圈就能自动校准罗盘的高级功能
- 减少“罗盘受到干扰”的报错，提高航向的精准度，不需要每次飞之前都重新校准罗盘

```
# Pre-takeoff continuous magnetometer calibration
if param compare -s MBE_ENABLE 1
then
    mag_bias_estimator start
fi
```

- 板级专用 MAVLink 服务的自动加载
- 有些飞控的板子上会有需要MAVLink通信的外设，通过执行`rc.board_mavlink`文件，来配置这些外设的MAVLink通信

```
#
# Optional board mavlink streams: rc.board_mavlink
#
set BOARD_RC_MAVLINK ${R}etc/init.d/rc.board_mavlink
if [ -f $BOARD_RC_MAVLINK ]
then
    echo "Board mavlink: ${BOARD_RC_MAVLINK}"
    . $BOARD_RC_MAVLINK
fi
unset BOARD_RC_MAVLINK
```

- 启动串口驱动
- `rc.serial`根据在QGC中设置的参数，把飞控上的物理串口分配给对应的功能

```
#
# Start UART/Serial device drivers.
# Note: rc.serial is auto-generated from Tools/serial/generate_config.py
#
. ${R}etc/init.d/rc.serial
```

- 遥控器的输入必须在串口配置完之后启动
- 因为先要确定哪些串口被占用了之后，用剩下的串口来扫描接收机信号，或者直接指定串口

```
# Must be started after the serial config is read
rc_input start $RC_INPUT_ARGS
```

- 管理USB接口
- 飞控上的USB在Nuttx系统里被识别为`ttyACM0`
- 每次将飞控连上电脑，是启动了一个MAVLink数据流，专门用来和地面站通信

```
# Manages USB interface
if param greater -s SYS_USB_AUTO -1
then
    if ! cdcacm_autostart start
```

```

        then
            sercon
            echo "Starting MAVLink on /dev/ttyACM0"
            mavlink start -d /dev/ttyACM0
        fi
    fi
fi

```

- 播放开机启动音效
- 如果启动失败，播放error音效
- **CBRK_BUZZER 782090**当把这个参数设为这个值时，禁用蜂鸣器
- 如果是报错音，会无视静音设置强制播放

```

#
# Play the startup tune (if not disabled or there is an error)
#
param compare CBRK_BUZZER 782090
if [ "$?" != "0" -o "$STARTUP_TUNE" != "1" ]
then
    tune_control play -t $STARTUP_TUNE
fi

```

- 启动导航模块
- Commander -> Navigator -> Position Controller

```

#
# Start the navigator.
#
navigator start

```

- 检查并运行温度校准
- 只有在在QGC中修改参数才能使用该功能

```

#
# Start a thermal calibration if required.
#
set RC_THERMAL_CAL ${R}etc/init.d/rc.thermal_cal
if [ -f ${RC_THERMAL_CAL} ]
then
    . ${RC_THERMAL_CAL}
fi
unset RC_THERMAL_CAL

```

- 启动云台驱动

```
#
# Start gimbal to control mounts such as gimbals, disabled by default.
#
if param greater -s MNT_MODE_IN -1
then
    gimbal start
fi
```

- 黑羊遥测
- 老技术

```
# Blacksheep telemetry
if param compare -s TEL_BST_EN 1
then
    bst start -X
fi
```

- 重要!
- 陀螺仪 FFT 频谱分析
- 在飞行中实时分析陀螺仪数据里的频率
- 出来的频率数据会直接喂给动态陷波滤波器，滤掉噪声，减少电机发热等

```
if param compare -s IMU_GYRO_FFT_EN 1
then
    gyro_fft start
fi
```

- 陀螺仪在线校准
- 启动运行时陀螺仪偏置校准
- 这不同于在地面站做的静态校准。这是一个在某些特定条件下运行的辅助校准逻辑，用于在长时间运行中修正漂移

```
if param compare -s IMU_GYRO_CAL_EN 1
then
    gyro_calibration start
fi
```

- 检查PX4Flow 光流传感器
- 启动老款的 PX4Flow 光流模块驱动
- 用于室内无 GPS 环境下的定位

```
# Check for px4flow sensor
if param compare -s SENS_EN_PX4FLOW 1
```

```
then
    px4flow start -X &
fi
```

- 载荷投送
- 启动抛投器/夹爪控制模块
- 负责控制舵机打开钩子，或者控制磁铁断电扔下包裹

```
payload_deliverer start
```

- 内燃机控制
- 启动油动发动机控制逻辑
- 用于油动无人机

```
if param compare -s ICE_EN 1
then
    internal_combustion_engine_control start
fi
```

- 可选的板载附加组件
- 用于处理既不是传感器，也不是MAVLink通信的外设
- 比如说接一个OLED屏幕等等
- `rc.board_extras`扩展脚本
- 修改该PX4源码，需要重新编译和烧录
- 优点：稳定

```
#
# Optional board supplied extras: rc.board_extras
#
set BOARD_RC_EXTRAS ${R}etc/init.d/rc.board_extras
if [ -f $BOARD_RC_EXTRAS ]
then
    echo "Board extras: ${BOARD_RC_EXTRAS}"
    . $BOARD_RC_EXTRAS
fi
unset BOARD_RC_EXTRAS
```

- 加载SD卡上的自定义扩展脚本
- 启动那些官方固件里没有、但是自己写在SD卡里的程序
- 和`config.txt`有点区别
- 如果有扩展的外设，建议写在这个文件中，而不是上面的`rc.board_extras`
- 不需要重新编译，从SD卡中启动
- Nuttx Shell脚本，和rcS类似
- 缺点：SD故障会丢失
- 除非是很冷门的外设，一般都不需要写这个脚本，而是在QGC里面调参数来开启或者关闭外设

```
#
# Start any custom addons from the sdcard.
#
if [ -f $FEXTRAS ]
then
    echo "Addons script: ${FEXTRAS}"
    . $FEXTRAS
fi
```

- 启动日志系统
- "行车记录仪"

```
#
# Start the logger.
#
set RC_LOGGING ${R}etc/init.d/rc.logging
if [ -f ${RC_LOGGING} ]
then
    . ${RC_LOGGING}
fi
unset RC_LOGGING
```

- 这里也是机架配置
- 配置所有机架通用的配置
- 这些通用配置写在另外的文件里，而不是在每个机架文件里重复写

```
#
# Set additional parameters and env variables for selected AUTOSTART.
#
if ! param compare SYS_AUTOSTART 0
then
    . ${R}etc/init.d/rc.autostart.post
fi
```

- Bootloader 自动升级机制
- 检查当前的 PX4 固件里是否打包了新版本的 Bootloader，如果有就运行升级脚本来升级 Bootloader
- Bootloader 功能：检测 USB 有没有插着，如果有，就允许通过 QGC 刷入新固件；如果没有，就启动 PX4 固件

```
set BOARD_BOOTLOADER_UPGRADE ${R}etc/init.d/rc.board_bootloader_upgrade
if [ -f $BOARD_BOOTLOADER_UPGRADE ]
then
    sh $BOARD_BOOTLOADER_UPGRADE
```

```
fi
unset BOARD_BOOTLOADER_UPGRADE
```

- 启动CAN总线
- 下面这两个二选一
- UAVCAN：旧版 UAVCAN v0
- Cyphal：新版 UAVCAN v1
- 是否开启Zenoh
- Zenoh：网络协议，主要用于机器人领域（ROS 2）。它不仅仅是连电调的，更多是用来连接机载电脑、进行高吞吐量的云端通讯或者边缘计算通讯。它是独立的。不管你用不用 CAN，只要你想用 Zenoh 连网，它就会启动

```
#
# Check if UAVCAN is enabled, default to it for ESCs.
#
if param greater -s UAVCAN_ENABLE 0
then
    # Start core UAVCAN module.
    if ! uavcan start
    then
        tune_control play error
    fi
else
    if param greater -s CYPHAL_ENABLE 0
    then
        cyphal start
    fi
fi
if param greater -s ZENOH_ENABLE 0
then
    zenoh start
fi

#
# End of autostart.
#
fi
```

- 启动完成!

三.通信协议

- PX4内部通过uORB进行通信
- PX4和外界通过MAVLikn进行通信

1.uORB

- uORB 是 PX4 内部的异步消息传输机制（IPC，进程间通信）

- 传感器驱动只管发数据，不需要知道谁在用数据
- 工作方式：布/订阅模式 (Publish / Subscribe)
- Topic (话题)：从本质上讲，一个 Topic 就是一个结构体 (Struct)，定义在 .msg 文件中（例如 vehicle_attitude.msg）
- Node (节点)：任何一个后台进程 (module) 都可以是发布者 (Advertiser) 或订阅者 (Subscriber)。

2.MAVLink

- MAVLink 是一种轻量级的通信协议，用于飞控与外部世界（地面站 QGC、机载电脑、OSD、数传）进行交互。
- 物理层：通常运行在 串口 (UART/Serial) 或 UDP/TCP 网络上。
- 逻辑层：它定义了一套消息 ID 和 Payload 格式。
- 飞控内部只认 uORB，外部只认 MAVLink。它们之间通过 mavlink 模块 进行转换

-
- 举例：

-
- 飞控 -> 地面站：
 - mavlink 模块订阅 uORB 的 vehicle_attitude
 - 收到更新后，打包成 MAVLink 的 ATTITUDE 消息包
 - 通过串口发送出去

-
- 地面站 -> 飞控：
 - mavlink 模块从串口收到 COMMAND_LONG (比如起飞指令)
 - 解析后，将其转换为 uORB 的 vehicle_command 消息并发布
 - commander 模块订阅到该指令并执行

四.飞控算法

五.感知导航